

1. File Systems Basics (Lecture 1)

، يعني بيحتفظ بالبيانات حتى Persistent Storage يعتبر الـ Disk. الـ Disk المسؤول عن إدارة الـ OS هو الجزء من الـ **Filesystem** الـ بعد ما نفصل الكهريا

Hardware vs. Software Storage Units:

- **Sector:** The physical unit of storage on a hard disk. Hard disks are sector-addressable, meaning you can only read/write in chunks of sectors, not single bytes.
- **Block:** The logical unit of data defined by the filesystem (software abstraction). A block consists of 1 or more sectors.
 - *Exam Trade-off:* Larger block size = more efficient I/O, but more **Internal Fragmentation**. Smaller block size = less internal fragmentation.

Types of Storage :

- **Hard Drives (Magnetic Disks):** Use spinning magnetic platters and an actuator arm with heads to read/write. The main performance bottleneck is "seeking" (moving the arm and waiting for the disk to spin). Moving parts risk damage.
- **Flash Storage (SSD):** No moving parts (more reliable), ~100x faster access.
 - *Exam Note (Constraint):* SSDs have wear-out issues; a chunk can only be erased ~100k times before it becomes unreliable.

What data do we store?

- **Payload Data:** The actual contents of the files (text, pixels, etc.).
- **Metadata:** Information *about* the files (name, type, size, extension). Both must be stored on disk to persist across power-offs.

2. File Allocation Methods (Exam Comparison)

دي مقارنة بين الـ ٣ طرق الأساسية ؟ ده سؤال امتحان مضمون Disk بتاعة الملفات على الـ Blocks إزاي بنرتب الـ

Allocation Method	Description	Advantages	Disadvantages
Contiguous	File is allocated in one contiguous group of blocks.	Simple. Supports Sequential and Random Access easily. Few seeks (fast).	Hard to grow files. Suffers from External Fragmentation .
Linked List	Each block contains file data + a pointer (location) to the next block.	Easy to grow files. Less fragmentation. Supports sequential access.	Cannot easily jump to arbitrary locations (No random access). More seeks (scattered data).

Allocation Method	Description	Advantages	Disadvantages
FAT (File Allocation Table)	Like linked, but links are stored in a centralized table in memory/disk instead of inside the blocks.	Can jump to locations more quickly. Keeps easy sequential access.	Data is still scattered (more seeks). The FAT table must be stored in memory.

💡 **Exam Definition - Fragmentation:** * **Internal Fragmentation:** Space allocated for a file is larger than what is actually needed (e.g., file is 300 bytes, but block is 512 bytes, wasting 212 bytes). * **External Fragmentation:** Enough aggregate free space exists on the disk, but no *single contiguous space* is large enough to satisfy an allocation request.

3. Multi-level Index & Unix v6 Inodes (Lecture 2)

Inodes والـ **Trees** استخدم فكرة الـ Unix v6 عشان نحل مشاكل الطرق اللي فاتت، نظام

What is an Inode?

An **inode (index node)** is a 32-byte data structure on disk that stores a file's metadata and an ordered list of block numbers containing the file's payload data.

- In Unix v6, the inode table starts contiguously at block 2.
- If 1 block = 512 bytes, we can fit 16 inodes per block.

Addressing Modes in Unix v6 :

The inode uses an array called `i_addr` with exactly **8 entries** to store block numbers. Because 8 blocks aren't enough for large files, Unix v6 uses two modes (Small and Large):

1. Small Mode (Direct Addressing):

- All 8 entries in `i_addr` point directly to data blocks.
- *Max File Size:* $8 \times 512 = 4096$ bytes (≈ 4 KB).

2. Large Mode (Indirect Addressing):

To support larger files, pointers don't point to data, they point to blocks that contain *more* pointers!

- **Singly-Indirect:** The pointer points to a block full of pointers. If a block is 512 bytes and a pointer is 2 bytes, one indirect block holds 256 pointers.
 - *Max Size with 8 Singly-Indirect:* $8 \times 256 \times 512 \approx 1$ MB.
- **Doubly-Indirect:** To go even bigger, the first 7 entries are Singly-Indirect, and the 8th entry is Doubly-Indirect (points to a block of pointers, which point to blocks of pointers).
 - *Max Size Calculation:* $(7 + 256) \times 256 \times 512 \approx 34$ MB.

 **Exam Example / Practice:**

Question: What is the smallest file size that requires the doubly-indirect block? *Answer:* Files up to $(7 \times 256 \times 512)$ bytes can be represented using just the 7 singly-indirect blocks. Therefore, a file of $(7 \times 256 \times 512) + 1$ bytes or more will need the 8th doubly-indirect block.

4. Directories & Lookups (Lecture 3)

كيف نعرف يوصل لملف زي OS إزاي الـ /classes/hello.txt؟

- **Directories as Files:** In Unix, a directory is basically treated as a file. It has its own inode.
- **Directory Payload:** The contents of a directory are simply an unsorted list of "Directory Entries".
- **Directory Entry Structure:** Each entry is 16 bytes:
 - **2 bytes:** The inumber (Inode Number).
 - **14 bytes:** The file/folder name.

The Lookup Process (Step-by-Step Traversal):

If you want to access /classes/operating-systems/hello.txt:

1. **Start at Root (/):** The root directory always has an inumber of **1**. (Note: inumber 0 is reserved to mean "NULL").
2. Go to inode 1, read its data blocks to find the directory entries.
3. Search the entries for the name classes and get its inumber.
4. Go to that new inode, read its data, search for operating-systems, get its inumber.
5. Repeat until you find hello.txt, get its inumber, go to its inode, and finally read the payload data.